

BETHEL GENERAL INSURANCE AND SURETY CORPORATION
Legazpi Branch

CLIENT SERVICES PORTAL

Technical Documentation

Tech Stack | Dependencies | Deployment Guide

Version 1.0

June 2026

Bicol University – College of Science
BSIT Capstone Project

1. SYSTEM OVERVIEW

The Bethel Gen Client Services Portal is a full-stack web application that digitizes the insurance application process of Bethel General Insurance and Surety Corporation – Legazpi Branch. Clients can apply for policies, file claims, upload and track documents, receive notifications, and message the branch through a secure online portal. Branch staff manage all of this through a dedicated admin portal.

1.1 Project Information

Field	Details
System Name	Digitizing Client Services: An Application Portal for Bethel General Insurance and Surety Corporation – Legazpi Branch
Version	1.0 (Prototype / Demo)
Project Type	BSIT Capstone Project
Institution	Bicol University – College of Science, Legazpi City, Albay
Client	Bethel General Insurance and Surety Corporation – Legazpi Branch
Development Period	2025 – 2026

1.2 System Portals

Portal	Description
Landing Page	Public page with company info, products, Get a Quote flow, and login/register access.
Client Portal	Authenticated area for clients: manage applications, upload documents, view AI validation, receive notifications, and message the branch.
Admin Portal	Authenticated area for branch staff: review applications and documents, send messages, manage calendar tasks, and view reports.

2. FINAL TECH STACK

Layer	Technology
Backend Framework	Laravel 9 (PHP)
Frontend Framework	React 18 with Vite 5
Database	MySQL 8.x
Authentication	Laravel Sanctum (Bearer token)

Real-time Messaging	Polling (3-second interval) — Pusher-ready
AI Document Validation	OpenAI-compatible LLM API (optional)
HTTP Client (Frontend)	Axios ^1.7.2
HTTP Client (Backend)	Guzzle ^7.2
Local Dev Environment	XAMPP (Apache + MySQL + PHP)

3. DEPENDENCIES AND VERSIONS

3.1 Backend (PHP / Composer)

Package	Version	Purpose
PHP	^8.0 (tested: 8.2+)	Runtime language
laravel/framework	9.52.16	Core MVC framework
laravel/sanctum	^3.0	API token authentication
laravel/tinker	^2.7	Interactive PHP console
guzzlehttp/guzzle	^7.2	HTTP client for LLM API calls
pusher/pusher-php-server	^7.2	Broadcast driver (optional)
fakerphp/faker	^1.9.1	Dev: fake data for testing
phpunit/phpunit	^9.5.10	Dev: unit testing

3.2 Frontend (Node.js / npm)

Package	Version	Purpose
Node.js	20 LTS (minimum)	JavaScript runtime
react	^18.3.1	UI component library
react-dom	^18.3.1	React DOM renderer
react-router-dom	^6.23.1	Client-side page routing
axios	^1.7.2	HTTP requests to Laravel API
lucide-react	^0.383.0	Icon library
laravel-echo	^1.15.3	WebSocket client (Pusher-ready)
pusher-js	^8.4.0-rc2	Pusher WebSocket (optional)
vite	^5.2.0	Frontend build tool and dev server

@vitejs/plugin-react	^4.3.1	Vite plugin for React/JSX
----------------------	--------	---------------------------

4. PREREQUISITES

Install the following software on the machine before running the system.

Software	Minimum Version	Download
XAMPP	8.2+ (includes PHP 8.2)	https://www.apachefriends.org
Composer	2.x (latest)	https://getcomposer.org
Node.js	20 LTS	https://nodejs.org
Git	Any recent version	https://git-scm.com

PHP Extensions Required. After installing XAMPP, open `C:\xampp\php\php.ini` and make sure these lines are active (no leading semicolon):

```
extension_dir = "C:\xampp\php\ext"
extension=curl
extension=fileinfo
extension=mbstring
extension=openssl
extension=pdo_mysql
extension=zip
```

5. PROJECT FOLDER STRUCTURE

```
bethel-system/
├── backend/                                Laravel 9 API
│   ├── app/Http/Controllers/              All API controllers
│   ├── app/Models/                        Eloquent models
│   ├── app/Services/                      AiService (LLM)
│   ├── config/                           App configuration
│   ├── database/migrations/               Table definitions
│   ├── database/seeder/                   Demo data
│   ├── routes/api.php                     All REST routes
│   ├── storage/app/private/                Uploaded documents
│   ├── .env.example                       Config template
│   └── composer.json                      PHP dependencies
└── frontend/                             React + Vite
```

public/	PNG logos and icons
src/pages/	All page components
src/context/	Auth state
src/utils/	api.js, echo.js
.env.example	Config template
package.json	Node dependencies
docs/	Documentation

6. ENVIRONMENT CONFIGURATION

6.1 Backend (.env)

Variable	Value / Description
APP_KEY	Auto-generated by: php artisan key:generate
DB_DATABASE	bethel_portal
DB_USERNAME	root
DB_PASSWORD	(blank for XAMPP default)
BROADCAST_DRIVER	log (use pusher only if enabling real-time)
PUSHER_APP_ID / KEY / SECRET	From pusher.com — optional
PUSHER_APP_CLUSTER	ap1 (Asia Pacific) or mt1
LLM_API_KEY	OpenAI or Groq key — optional for AI validation
LLM_MODEL	gpt-4o-mini or llama3-8b-8192 (Groq free tier)
LLM_BASE_URL	https://api.openai.com/v1 or https://api.groq.com/openai/v1
FRONTEND_URL	http://localhost:5173

6.2 Frontend (.env)

Variable	Value
VITE_PUSHER_APP_KEY	Same as backend (or dummy value if not using real-time)
VITE_PUSHER_APP_CLUSTER	ap1 or mt1

7. DEMO CREDENTIALS

Role	Email	Password
Admin	admin@bethelgen.com	Admin@1234
Client (Demo)	juan@gmail.com	Client@1234

Login at: <http://localhost:5173/login> — toggle between Client and Admin tabs before entering credentials.

8. RUNNING THE SYSTEM LOCALLY

Open **two separate CMD windows** — one for the backend, one for the frontend.

Step 1 — Start XAMPP

1. Open XAMPP Control Panel.
2. Start Apache and MySQL.
3. Open phpMyAdmin: <http://localhost/phpmyadmin>
4. Create a database named:
`bethel_portal`

Step 2 — Backend (CMD Window 1)

5. Navigate to the backend folder:
`cd bethel-system\backend`
6. Install PHP dependencies:
`composer install --ignore-platform-reqs`
7. Copy and configure the .env file:
`copy .env.example .env`
`notepad .env`
Fill in DB credentials. Save and close.
8. Generate the application key:
`php artisan key:generate`
9. Run migrations and seed demo data:
`php artisan migrate --seed`

Note: If you see a duplicate entry error, run: `php artisan migrate:fresh --seed`

Note: If migrating the database via export/import (see Section 9), skip this step entirely.

10. Create storage folders (first time only):
`mkdir storage\app\private`
`mkdir storage\app\private\documents`
11. Create the storage symlink:

```
php artisan storage:link
```

12. Start the Laravel server:

```
php artisan serve
```

API runs at: <http://localhost:8000> — keep this window open.

Step 3 — Frontend (CMD Window 2)

13. Navigate to the frontend folder:

```
cd bethel-system\frontend
```

14. Install Node.js dependencies (first time only):

```
npm install
```

15. Copy and configure the .env file:

```
copy .env.example .env
```

16. Start the Vite development server:

```
npm run dev
```

Portal runs at: <http://localhost:5173> — keep this window open.

9. MIGRATING TO ANOTHER MACHINE

There are three methods. **Method A (Database Export) is the recommended and fastest approach** as it preserves all existing data — including demo applications, messages, documents, and settings — without running seeders.

Method A — Database Export / Import (Recommended)

What this does: Exports your entire database as a .sql file and imports it on the new machine. Skips migrations and seeding entirely — all existing data is preserved.

Part 1 — On the source machine (export):

17. Open phpMyAdmin:

```
http://localhost/phpmyadmin
```

18. Click the

bethel_portal database on the left.

19. Click the

Export tab at the top.

20. Export method:

Quick — click **Go**. Save the file (e.g. `bethel_portal.sql`).

21. Copy the project folder and the .sql file to a USB drive.

Exclude these folders from the project copy (large, auto-reinstalled):

- `bethel-system\backend\vendor\`

- bethel-system\frontend\node_modules\

Part 2 — On the new machine (import):

22. Install all Prerequisites from Section 4.
23. Open XAMPP → start Apache and MySQL.
24. Open phpMyAdmin → create empty database:
bethel_portal
25. Click the
bethel_portal database → click **Import** tab → choose the .sql file → click **Go**.
26. Copy the project folder to the machine. Run backend and frontend setup:

```
cd bethel-system\backend
composer install --ignore-platform-reqs
copy .env.example .env
notepad .env      ← fill in DB credentials + APP_KEY placeholder
php artisan key:generate
mkdir storage\app\private
mkdir storage\app\private\documents
php artisan storage:link
php artisan serve

cd ../frontend
npm install
copy .env.example .env
npm run dev
```

Note: Do NOT run php artisan migrate --seed — the database is already populated from the import.

Method B — USB / Flash Drive (Simple, Fresh Start)

Use this if you want a clean install with only the seeded demo data.

On the source machine:

27. Copy the project folder to USB, excluding
vendor\ and node_modules\.

On the new machine:

28. Install all Prerequisites (Section 4).
29. Copy project from USB.
30. Follow the full setup in Section 8 — including
php artisan migrate --seed.

Method C — Git Repository

On the source machine (push):

Create .gitignore in backend\:

```
/vendor/  
.env  
/storage/app/private/  
/bootstrap/cache/
```

And in frontend\:

```
node_modules/  
dist/  
.env
```

31. Push to GitHub:

```
git init && git add .  
git commit -m "Bethel Gen Portal"  
git remote add origin https://github.com/your-username/bethel-system.git  
git push -u origin main
```

On the new machine (clone):

32. Clone the repo:

```
git clone https://github.com/your-username/bethel-system.git
```

33. Follow full setup in Section 8.

Note: .env files are excluded from Git. Manually create them on each machine from .env.example.

10. COMMON ERRORS AND FIXES

Error	Fix
"composer" not recognized	Install Composer from getcomposer.org. Restart CMD after install.
"npm" not recognized	Install Node.js from nodejs.org. Restart CMD after install.
PHP extension errors (curl, mbstring, etc.)	Open php.ini, set extension_dir to your PHP ext folder, and uncomment the required extensions (Section 4).
SQLSTATE[HY000] Access denied	Set DB_PASSWORD= (blank) in .env for default XAMPP.
"Duplicate entry" on seeder	Run: php artisan migrate:fresh --seed
500 Internal Server Error on API	Check backend CMD window for the exact PHP error. Common causes: .env not configured, missing APP_KEY.
CORS error in browser	Set FRONTEND_URL=http://localhost:5173 in backend .env. Restart php artisan serve.
File upload error / file appears after refresh	Run: mkdir storage\app\private\documents in the backend folder.

"Route [login] not defined"	Check app\Http\Middleware\Authenticate.php — redirectTo() must return null.
PowerShell execution policy error	Run as Administrator: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
Vite blank page after npm run dev	Hard-refresh: Ctrl + Shift + R. Check browser F12 Console for JS errors.

11. QUICK COMMAND REFERENCE

Command	What it does
php artisan serve	Start Laravel API at localhost:8000
php artisan migrate --seed	Create tables and insert demo data
php artisan migrate:fresh --seed	Drop all tables, recreate, and reseed
php artisan key:generate	Generate APP_KEY in .env
php artisan storage:link	Create public storage symlink
php artisan config:clear	Clear cached config
composer install --ignore-platform-reqs	Install PHP dependencies
composer dump-autoload --ignore-platform-reqs	Rebuild class autoloader
npm install	Install Node.js dependencies
npm run dev	Start Vite dev server at localhost:5173
npm run build	Build production-ready frontend

12. SYSTEM FEATURES SUMMARY

Landing Page

- Company information with tabs: Home, Products, News & Events, Partners, About Us
- Get a Quote modal — 3-step flow with product-specific property fields
- Sign In button with Client / Admin toggle and Register link

Client Portal

- Policy applications and claims filing
- Claims validation: requires an approved policy reference number
- Document uploads with automatic AI validation (score, issues, suggestions)
- Inline document viewer: image preview, PDF download
- Delete documents with confirmation modal and error handling
- Notifications with detail modal and contextual action buttons
- FAQ with AI-powered chatbot assistant
- Floating messaging widget with thread history

Admin Portal

- Dashboard with summary statistics
- Application management: Policies and Claims with search and filter
- Document review: view, download, approve or reject with feedback
- Status management, premium setting, and admin notes
- Calendar task manager with priority color-coding
- Two-panel messaging inbox
- Reports with monthly bar chart
- Notifications with detail modal and navigation actions

13. NOTES FOR CAPSTONE DEFENSE

Optional Features (Not Required for Demo)

- Pusher real-time: system uses 3-second polling. Enable by adding Pusher keys in both .env files.
- AI validation: requires OpenAI or Groq API key. Without one, uploads show as Pending Review — all other features work normally.
- Password reset emails: requires SMTP mail config. Feature is built but needs a mail service.

Demo Notes

- Runs entirely on localhost. No internet required once dependencies are installed.
- Uploaded documents are stored in backend/storage/app/private/documents/
- Tested on PHP 8.2.x and Node.js 20 LTS on Windows.